
TRÍ TUỆ NHÂN TẠO

Dữ liệu dạng số

Lý thuyết — Feature vector, chuẩn hóa, binning, scrubbing

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
1 Dữ liệu số (numerical data)	4
2 Cách mô hình nhập dữ liệu bằng vectơ đặc trưng	4
3 Các bước đầu tiên	6
3.1 Trực quan hóa dữ liệu	7
3.2 Đánh giá thống kê	7
3.3 Tìm outlier	7
4 Chuẩn hóa (normalization)	8
4.1 Linear scaling (chuẩn hóa tuyến tính)	9
4.2 Z-score scaling	10
4.3 Log scaling	11
4.4 Clipping	12
4.5 Bảng tóm tắt kỹ thuật chuẩn hóa	13
5 Binning (kết hợp / bucketing)	14
5.1 Ví dụ: số khách hàng vs nhiệt độ	15
5.2 Bài tập: giá nhà theo vĩ độ (Freedonia)	17
5.3 Quantile bucketing (bin theo phân vị)	17
6 Scrubbing (tua / làm sạch)	19
7 Tính chất của đặc trưng số tốt	20
7.1 Tên rõ ràng	20
7.2 Kiểm tra / thử trước khi train	20
7.3 Hợp lý (sensible) — tránh “magic value”	20
8 Biến đổi đa thức (polynomial transforms)	21
9 Kết luận	22

Lời nói đầu

Nguồn và cách đọc

Tài liệu biên soạn và dịch từ *Numerical Data* (Google Machine Learning Crash Course)^a.

^a<https://developers.google.com/machine-learning/crash-course/numerical-data>

Mục tiêu học tập

- Hiểu **feature vector** (vectơ đặc trưng).
- Khám phá đặc trưng tiềm năng bằng bảng trực quan và thống kê.
- Nhận diện **outlier** (giá trị ngoại lai).
- Nắm bốn kỹ thuật chuẩn hóa dữ liệu số phổ biến.
- Hiểu **binning** (bucketing) và chiến lược chia bin.
- Biết đặc tính của đặc trưng số liên tục tốt.

Điều kiện tiên quyết

Cần nắm khái niệm từ [Introduction to Machine Learning](#).

1 Dữ liệu số (numerical data)

Động cơ học – Tại sao cần khái niệm này?

Người làm ML thường dành **nhiều thời gian hơn** cho đánh giá, làm sạch và biến đổi dữ liệu so với xây mô hình.

Dữ liệu số

Dữ liệu số là số nguyên hoặc dấu phẩy động **có hành vi như số**: cộng được, đếm được, có thứ tự, v.v.

Ví dụ dữ liệu số

- Nhiệt độ
- Cân nặng
- Số hươu trú đông trong khu bảo tồn

Mã bưu điện Mỹ không phải dữ liệu số

Mã ZIP Mỹ dù 5 hoặc 9 chữ số nhưng **không** hành xử như số: 40004 (Nelson County, Kentucky) **không** phải “gấp đôi” 20002 (Washington, D.C.). Chúng biểu diễn **vùng địa lý** — là **dữ liệu danh mục**.

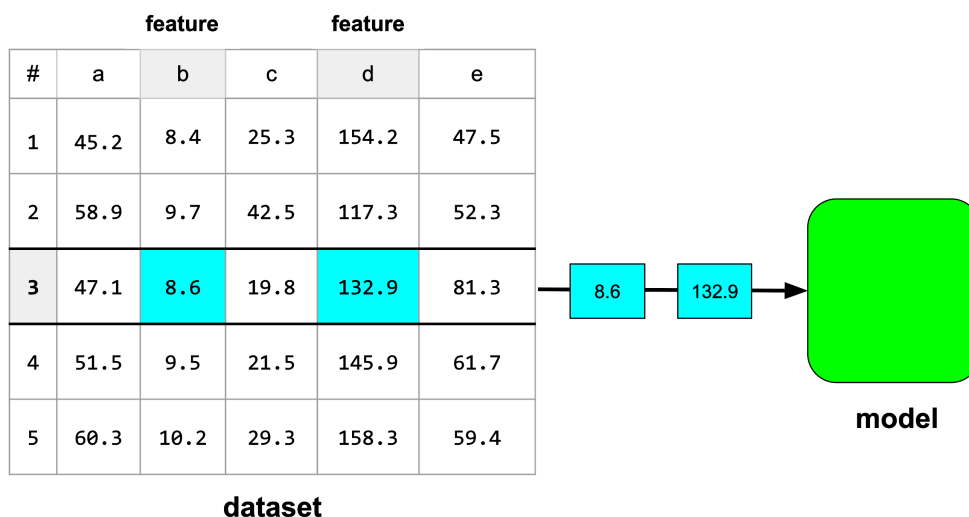
2 Cách mô hình nhập dữ liệu bằng vectơ đặc trưng

Ý nghĩa & Trực giác

Cho đến giờ có thể bạn tưởng mô hình đọc **trực tiếp** từng dòng bảng; thực tế mô hình nhận dữ liệu **khác**.

Dataset 5 cột, chỉ 2 cột là feature

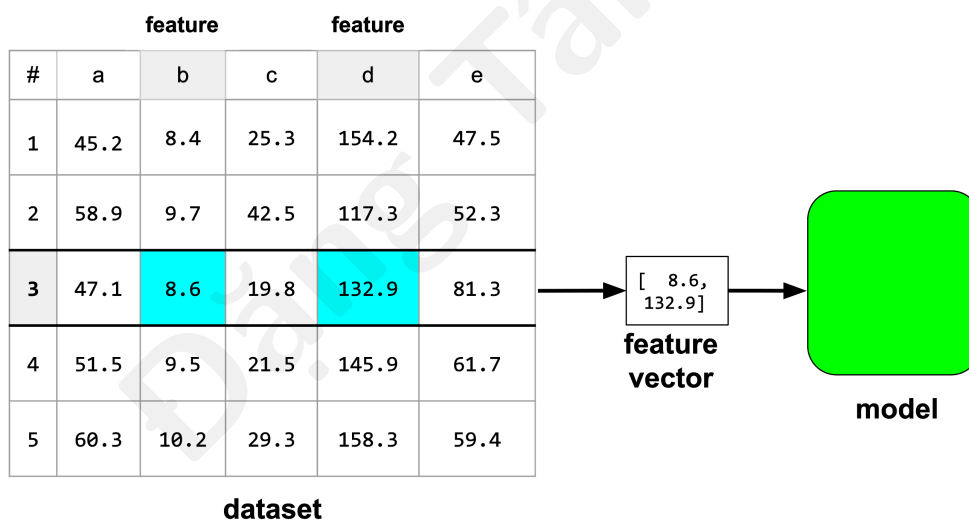
Giả sử bảng có 5 cột nhưng chỉ **b** và **d** đưa vào mô hình. Khi xử lý dòng 3, mô hình **không** chỉ lấy hai ô 3b và 3d như sau:



Hình 1. Mô hình lấy ví dụ trực tiếp từ dataset? — Không đúng.

Feature vector

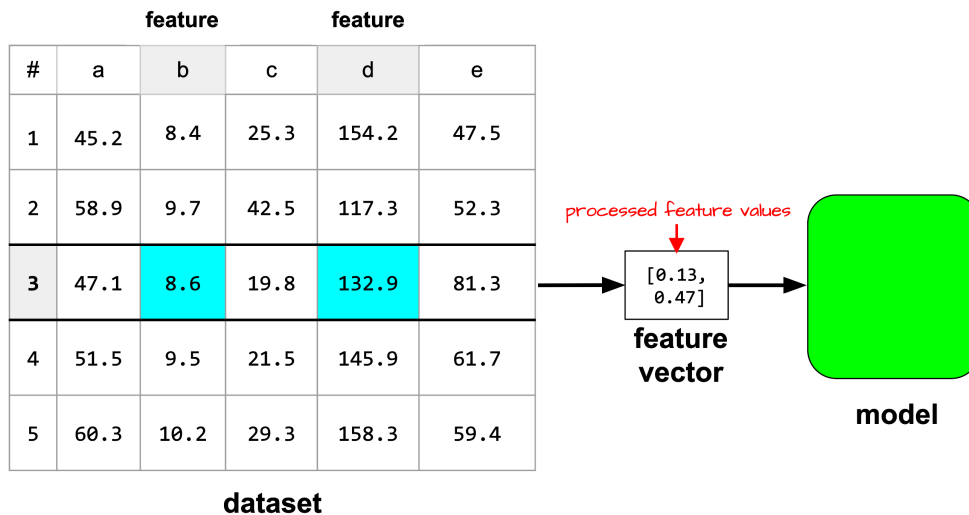
Mô hình thực sự nhận một mảng số thực (float) gọi là **feature vector** (vectơ đặc trưng) — các giá trị float tạo thành **một ví dụ**.



Hình 2. Feature vector là trung gian giữa dataset và mô hình — gần đúng hơn, nhưng chưa đủ thực tế.

Lưu ý

Feature vector **hiếm khi** dùng giá trị thô của dataset. Bạn phải xử lý giá trị thành biểu diễn mô hình học tốt hơn:



Hình 3. Feature vector thực tế (ví dụ 0,13 và 0,47) sau biến đổi.

Giá trị đã biến đổi thường tốt hơn giá trị thô

Dùng giá trị **gốc** trong dataset để huấn luyện có tốt hơn giá trị **đã sửa**? — **Không**. Bạn phải chọn cách biểu diễn giá trị thô thành giá trị huấn luyện trong feature vector. Quá trình này là **feature engineering** (kỹ thuật đặc trưng) — phần then chốt của ML.

Kỹ thuật feature engineering phổ biến nhất

- **Normalization** (chuẩn hóa): đưa giá trị số về cùng một miền quy mô.
- **Binning** / **bucketing**: gom giá trị số vào các khoảng (bin).

Lưu ý

Mọi phần tử feature vector phải là **float**. Nhiều đặc trưng tự nhiên là chuỗi hoặc không phải số; phần lớn feature engineering là biểu diễn giá trị **không phải số** thành số.

3 Các bước đầu tiên

Trước khi tạo feature vector

Nên nghiên cứu dữ liệu số theo **hai hướng**:

1. **Trực quan hóa** (biểu đồ, histogram).
2. **Thống kê** (mean, median, độ lệch chuẩn, phân vị).

3.1 Trực quan hóa dữ liệu

Ý nghĩa & Trực giác

Đồ thị giúp phát hiện bất thường hoặc mẫu ẩn. Nên xem dữ liệu dạng scatter plot hoặc histogram **ngay đầu pipeline** và **suốt** các bước biến đổi — trực quan giúp liên tục kiểm tra giả định.

Lưu ý

Khuyến nghị dùng pandas

3.2 Đánh giá thống kê

Ý nghĩa & Trực giác

Ngoài trực quan, nên tính thống kê cơ bản cho đặc trưng và nhãn tiềm năng:

- Trung bình (mean) và trung vị (median)
- Độ lệch chuẩn (standard deviation)
- Các phân vị: 0%, 25%, 50%, 75%, 100%. Phân vị 0% = min; 100% = max; 50% = median.

3.3 Tìm outlier

Outlier

Outlier là giá trị **xa** hầu hết giá trị khác trong một feature hoặc nhãn. Outlier thường gây hại khi huấn luyện.

Lưu ý

Khi khoảng (0% → 25%) khác **đáng kể** so với (75% → 100%), dataset có thể chứa outlier. **Đừng** chỉ dựa vào thống kê cơ bản — bất thường có thể ẩn trong dữ liệu “cân bằng” nhìn qua.

Ba loại outlier và cách xử lý

1. **Do lỗi** (thêm số 0, cảm biến hỏng...): thường **xóa** ví dụ chứa outlier lỗi.
2. **Hợp lệ, không phải lỗi** — mô hình cuối cùng có cần dự đoán tốt trên outlier đó không?
 - **Có**: giữ trong tập huấn luyện; outlier feature đôi khi trùng outlier nhãn và **giúp** dự đoán. Cẩn thận outlier cực đoan vẫn có thể hại mô hình.
 - **Không**: xóa outlier hoặc dùng kỹ thuật mạnh hơn như **clipping**.

4 Chuẩn hóa (normalization)

Động cơ học – Tại sao cần khái niệm này?

Sau khi xem dữ liệu bằng thống kê và trực quan, biến đổi sao cho mô hình huấn luyện hiệu quả hơn. Mục tiêu **chuẩn hóa**: đưa các feature về **cùng quy mô** tương đối.

Hai feature quy mô rất khác nhau

- Feature X: từ 154 đến 24 917 482.
- Feature Y: từ 5 đến 22.

Chuẩn hóa có thể đưa X và Y về cùng miền, ví dụ 0 đến 1.

Lợi ích chuẩn hóa

- Mô hình **hội tụ nhanh hơn**: feature khác miền khiến gradient descent “nảy”; optimizer như **Adagrad** hoặc Adam giảm bớt vấn đề bằng learning rate thích ứng.
- **Dự đoán tốt hơn** khi miền feature chênh lệch.
- Tránh **NaN trap**: giá trị vượt giới hạn float $\Rightarrow$ NaN, lan sang phần khác của mô hình.
- Mô hình học **trọng số phù hợp** cho từng feature thay vì chú ý quá mức feature miền rộng.

Lưu ý

Nên chuẩn hóa feature số có miền **khác hẳn nhau** (tuổi vs thu nhập) hoặc **một** feature phủ miền rất rộng (dân số thành phố).

Cảnh báo quan trọng

Nếu chuẩn hóa khi **train**, khi **suy luận (inference)** phải chuẩn hóa feature đó **theo cùng quy tắc** (cùng min/max, cùng μ , σ ...).

Feature miền hẹp nhưng chênh 10 lần

- Feature A: từ $-0,5$ đến $+0,5$.
- Feature B: từ $-5,0$ đến $+5,0$ (rộng gấp 10 lần A).

Lúc bắt đầu train, mô hình coi B quan trọng gấp 10; train lâu hơn; mô hình có thể kém tối ưu. Vẫn nên chuẩn hóa A và B về cùng quy mô (ví dụ -1 đến $+1$).

Feature miền chênh cực lớn

- C: -1 đến $+1$.
- D: $+5000$ đến $+1\,000\,000\,000$.

Không chuẩn hóa $\Rightarrow$ mô hình kém, train rất lâu hoặc **không hội tụ**.

4.1 Linear scaling (chuẩn hóa tuyến tính)

Định nghĩa

Linear scaling (thường gọi **scaling**) đưa float từ miền tự nhiên về miền chuẩn — thường 0–1 hoặc -1 – $+1$.

Công thức scale về 0–1

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

với x' giá trị đã scale, x giá trị gốc, $x_{\min}/x_{\max}$ min/max của feature trong dataset.

Feature quantity, miền 100–900, giá trị 300

$x = 300$, $x_{\min} = 100$, $x_{\max} = 900$:

$$x' = \frac{300 - 100}{900 - 100} = \frac{200}{800} = 0,25.$$

Khi nào dùng linear scaling?

Tốt khi **tất cả** điều kiện sau đúng:

- Biên dưới/trên dữ liệu ít đổi theo thời gian.
- Ít hoặc không có outlier cực đoan.
- Phân bố gần **đều** trên miền (histogram gần đều).

Ví dụ **age**: biên ≈ 0 – 100 ; outlier hiếm ($\approx 0,3\% > 100$); nhiều tuổi đều có đủ mẫu trong dataset lớn.

Lưu ý

Hầu hết feature thực tế **không** đủ điều kiện linear scaling; **Z-score** thường là lựa chọn tốt hơn.

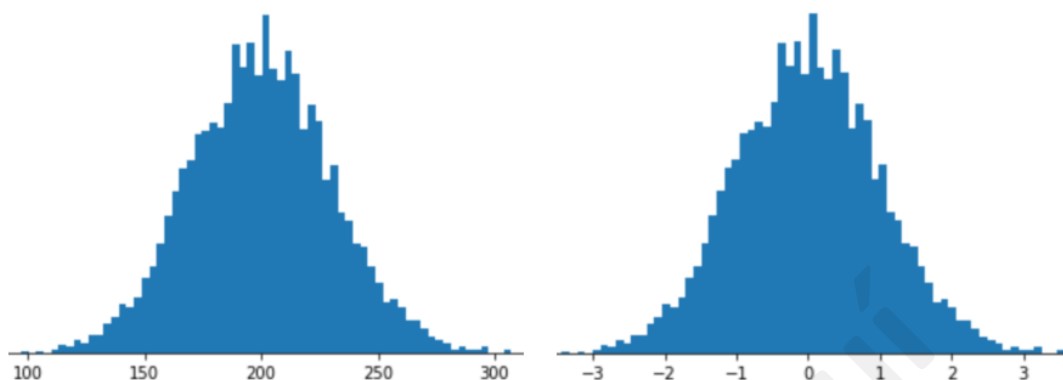
Bài tập: net_worth có nên linear scaling?

Đáp án: Không. Nhiều outlier; phân bố không đều — phần lớn người bị “ép” vào dải hẹp trong miền tổng thể.

4.2 Z-score scaling

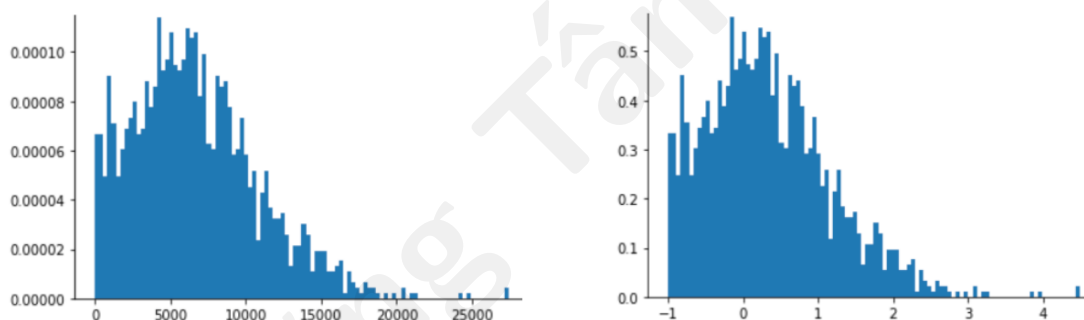
Định nghĩa

Z-score = số độ lệch chuẩn so với mean. Ví dụ: +2,0 nghĩa là lớn hơn mean 2 độ lệch chuẩn; -1,5 là nhỏ hơn 1,5 độ lệch chuẩn.



Hình 4. Phân bố chuẩn gốc (trái, mean ≈ 200 , $\sigma \approx 30$) vs Z-score (phải, mean 0, σ 1).

Z-score cũng phù hợp phân bố **gần** chuẩn nhưng không hoàn hảo:



Hình 5. Dữ liệu thô (0–29 000) vs Z-score (–1 đến $\approx +4,8$).

Công thức Z-score

$$x' = \frac{x - \mu}{\sigma}$$

(μ mean, σ độ lệch chuẩn.)

Ví dụ

Mean = 100, $\sigma = 20$, $x = 130$: $Z = (130 - 100)/20 = 1,5$.

Phân bố chuẩn cổ điển và Z-score hiếm

Trong phân bố chuẩn cổ điển: $\geq 68,27\%$ có $Z \in [-1, +1]$; $\geq 95,45\% \in [-2, +2]$; $\geq 99,73\% \in [-3, +3]$; $\geq 99,994\% \in [-4, +4]$. Điểm $|Z| > 4$ hiếm nhưng dataset cực lớn vẫn có thể có hàng chục nghìn ví dụ — “outlier” không có định nghĩa cứng.

Ý nghĩa & Trực giác

Z-score tốt khi dữ liệu theo phân bố chuẩn hoặc **gần** chuẩn. Một số feature chuẩn trong phần lớn miền nhưng vài ví dụ cách mean hàng trăm σ (`net_worth`) — kết hợp Z-score với **clipping**.

Bài tập: height (10 triệu phụ nữ trưởng thành)

Đáp án: Có. Gần phân bố chuẩn; 10 triệu mẫu $\Rightarrow$ đủ outlier để mô hình học mẫu Z rất cao/thấp.

4.3 Log scaling**Định nghĩa**

Log scaling tính logarit giá trị thô; thực hành thường dùng **ln** (log tự nhiên).

Công thức

$$x' = \ln(x)$$

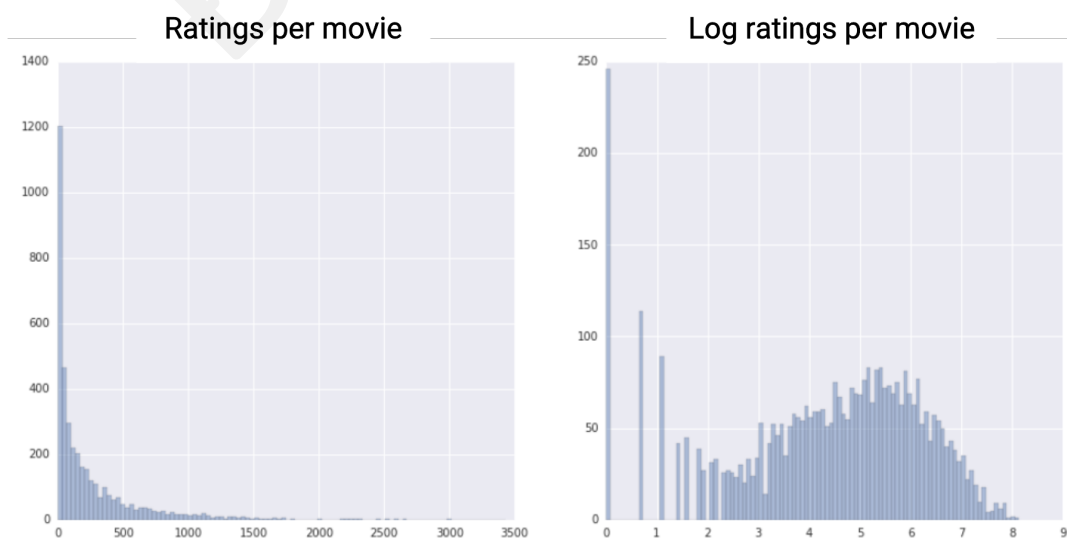
Ví dụ: $x = 54,598 \Rightarrow x' \approx 4,0$.

Phân bố lũy thừa (power law)

Nói nôm na: X nhỏ $\Rightarrow Y$ rất lớn; X tăng thì Y giảm nhanh; X lớn $\Rightarrow Y$ rất nhỏ.

Đánh giá phim: vài phim có rất nhiều rating; đa số có rất ít.

Bán sách: đa số bán vài trăm bản; một số vài nghìn; vài bestseller > 1 triệu. Mô hình tuyến tính trên giá trị thô phải tìm “sức mạnh” bìa sách triệu bản gấp 10 000 lần sách 100 bản — khó. Sau log: $\ln(100) \approx 4,6$, $\ln(10^6) \approx 13,8$ — chỉ gấp ≈ 3 lần, hợp lý hơn cho mô hình tuyến tính.



Hình 6. Phân bố thô (đầu dày, đuôi dài) vs sau log (phân bố đều hơn).

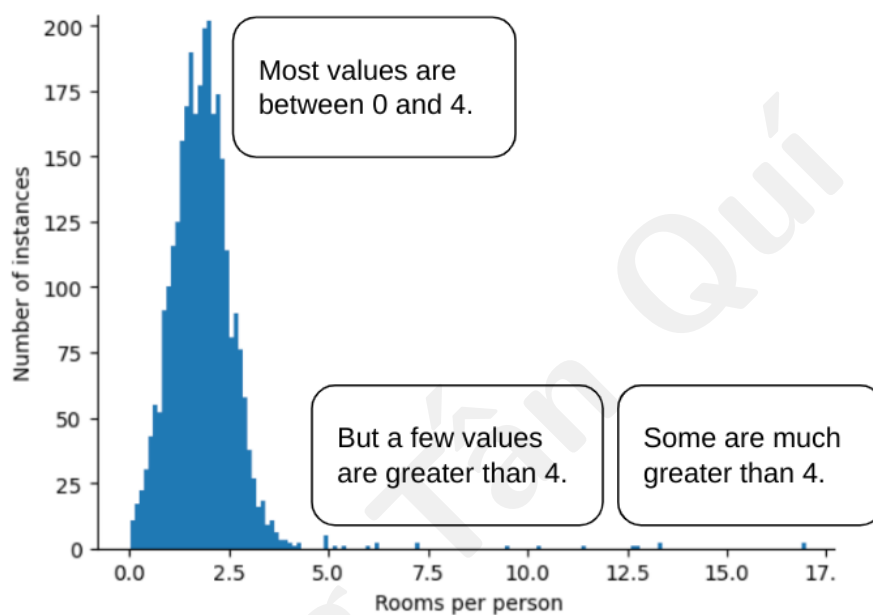
4.4 Clipping

Định nghĩa

Clipping giới hạn ảnh hưởng outlier cực đoan: thường **cap** giá trị outlier ở ngưỡng tối đa. Nghe lạ nhưng rất hiệu quả.

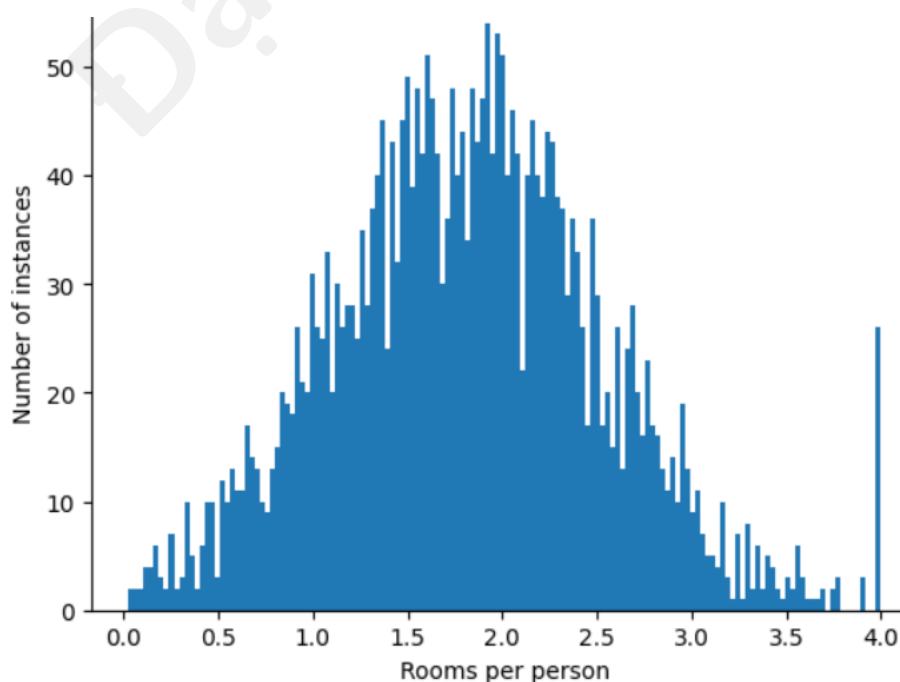
Feature roomsPerPerson

Số phòng chia số người ở mỗi nhà. Hơn 99% giá trị gần phân bố chuẩn (mean $\approx 1,8$, $\sigma \approx 0,7$) nhưng có outlier, một số cực đoan:



Hình 7. Phần lớn 0–4, đuôi dài đến 17 phòng/người.

Nếu **clip** tối đa roomsPerPerson ở 4,0:



Hình 8. Mọi giá trị trong $[0, 4]$; dính lại tại 4,0 do mọi giá trị > 4 thành 4 — vẫn hữu ích hơn dữ liệu gốc.

Ý nghĩa & Trực giác

Clip ở 4,0 **không** có nghĩa bỏ qua mọi giá trị > 4 ; nghĩa là chúng **đều thành** 4,0. Khi train mô hình, có thể clip tùy ý. Có thể clip **sau** Z-score: ví dụ clip $|Z| > 3$ về ± 3 . Clip cẩn thận — một số outlier thực sự quan trọng.

4.5 Bảng tóm tắt kỹ thuật chuẩn hóa

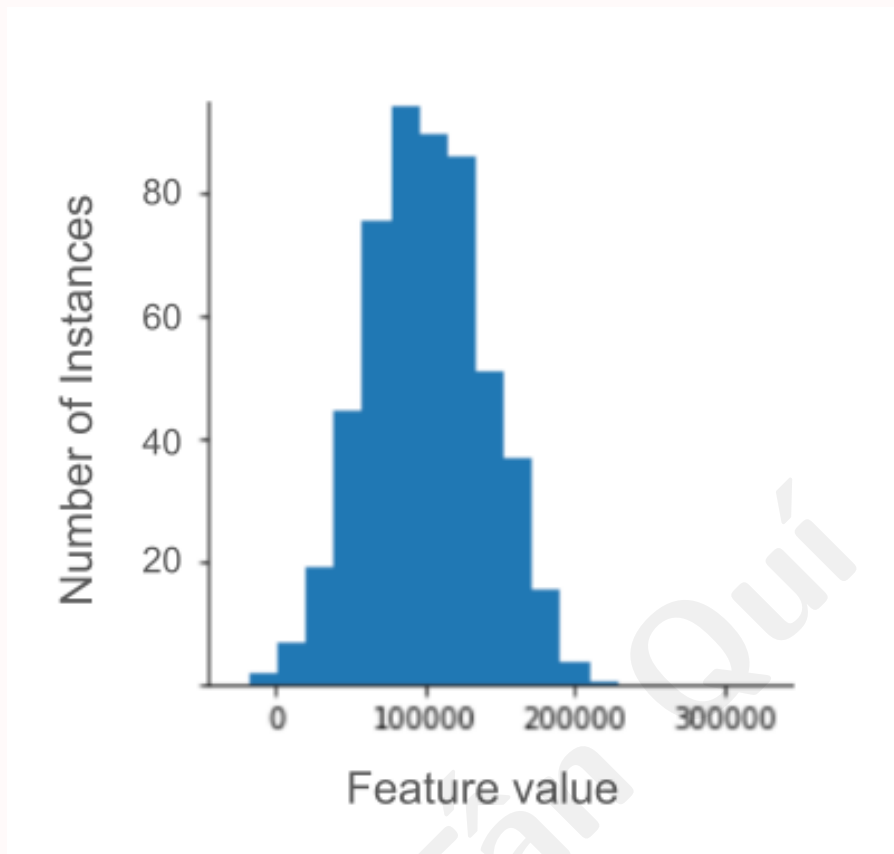
Lưu ý

Kỹ thuật tốt nhất là kỹ thuật **chạy tốt trên phân bố feature của bạn** — hãy thử ý tưởng mới nếu hợp lý.

Công thức

Kỹ thuật	Công thức	Khi dùng
Linear scaling	$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$	Phân bố đều trên miền (dạng phẳng).
Z-score	$x' = \frac{x - \mu}{\sigma}$	Phân bố gần chuông (đỉnh gần mean).
Log scaling	$x' = \ln(x)$	Đuôi dày , lệch mạnh (power law).
Clipping	Nếu $x > \max$ thì $x' = \max$; tương tự với min	Outlier cực đoan .

Bài tập: histogram 0–200 000, đỉnh quanh 100 000



Đáp án: Z-score scaling. Điểm gần phân bố chuẩn $\Rightarrow$ Z-score đưa về khoảng ≈ -3 đến $+3$.

Bài tập: nhiệt độ trung tâm dữ liệu 15–30 °C

Hầu hết 15–30. Ngoại lệ: vài ngày cực nóng 31–45 (hợp lệ); mỗi điểm thứ 1000 ghi nhầm 1000.

Đáp án hợp lý: Clip 31–45 (nếu ít mẫu nhiệt độ cao, model inference sẽ dự đoán giống nhau cho 35 và 45). **Xóa** các điểm 1000 (lỗi, không nên clip). Không nên xóa 31–45 hoặc clip 1000.

5 Binning (kết hợp / bucketing)

Binning

Binning (bucketing) là **feature engineering** gom các **khoảng con** số thành **bin**. Nhiều trường hợp biến số thành **danh mục**.

Feature X, min 15, max 425 $\rightarrow$ 5 bin

- Bin 1: 15–34 Bin 2: 35–117 Bin 3: 118–279
- Bin 4: 280–392 Bin 5: 393–425

Giá trị 17 và 29 đều ở Bin 1 — mô hình phản ứng **giống nhau** với cả hai.

Công thức

Số bin	Miền	Feature vector (one-hot)
1	15–34	[1, 0, 0, 0, 0]
2	35–117	[0, 1, 0, 0, 0]
3	118–279	[0, 0, 1, 0, 0]
4	280–392	[0, 0, 0, 1, 0]
5	393–425	[0, 0, 0, 0, 1]

Một cột **X** trong dataset sau binning được mô hình coi như **năm** feature; học **trọng số riêng** mỗi bin.

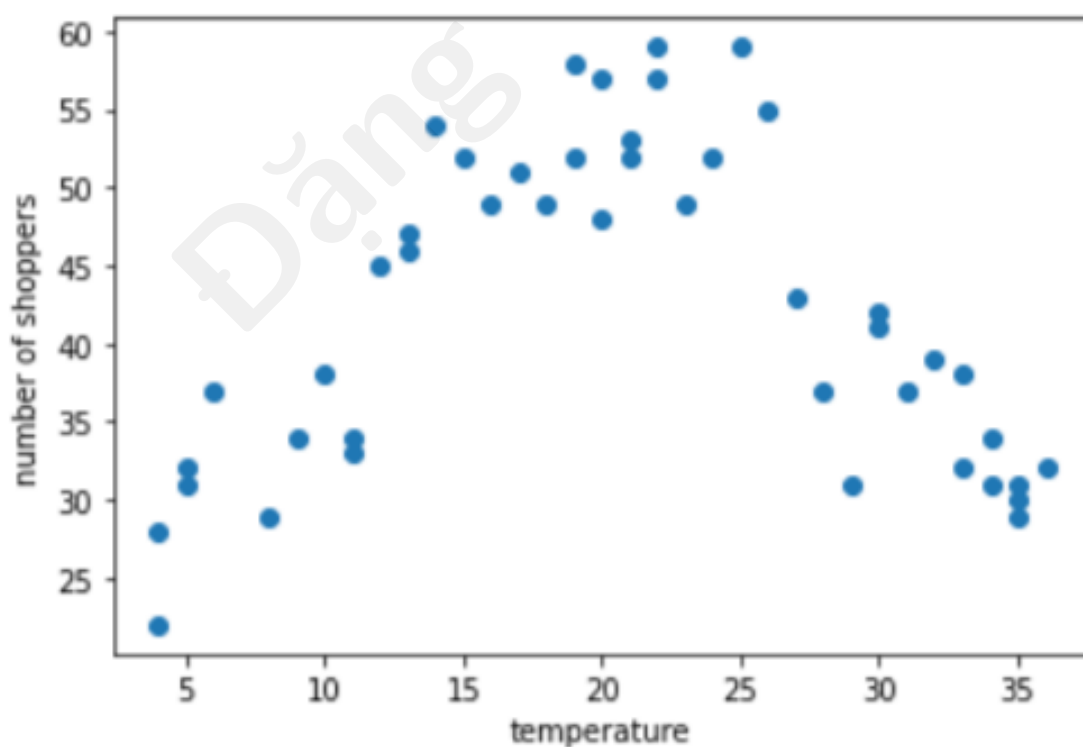
Khi binning tốt hơn scaling hoặc clipping

- Quan hệ **tuyến tính yếu hoặc không có** giữa feature và **nhân**.
- Giá trị feature **cụm** (clumpy) hơn là tăng tuyến tính.

37 và 115 có thể cùng bin — hợp lý khi dữ liệu “cục” hơn là đường thẳng.

5.1 Ví dụ: số khách hàng vs nhiệt độ

Mô hình dự đoán số người mua sắm theo nhiệt độ ngoài trời:

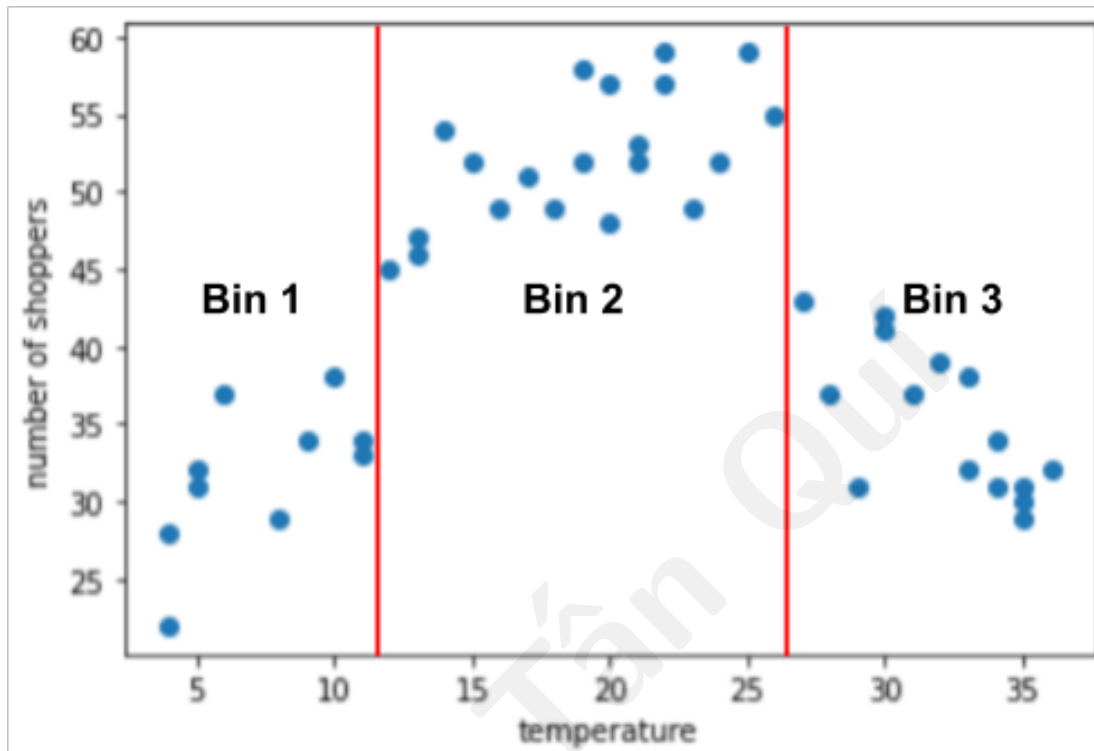


Hình 9. 45 điểm tự nhiên thành ba cụm.

Số khách cao nhất khi nhiệt độ **đễ chịu**. Nhiệt độ 35,0 trong feature vector (một số duy nhất) khiến hồi quy tuyến tính coi 35,0 ảnh hưởng gấp 5 lần 7,0 — trong khi đồ thị **không** tuyến tính.

Ba bin gợi ý:

- Bin 1: 4–11 Bin 2: 12–26 Bin 3: 27–36



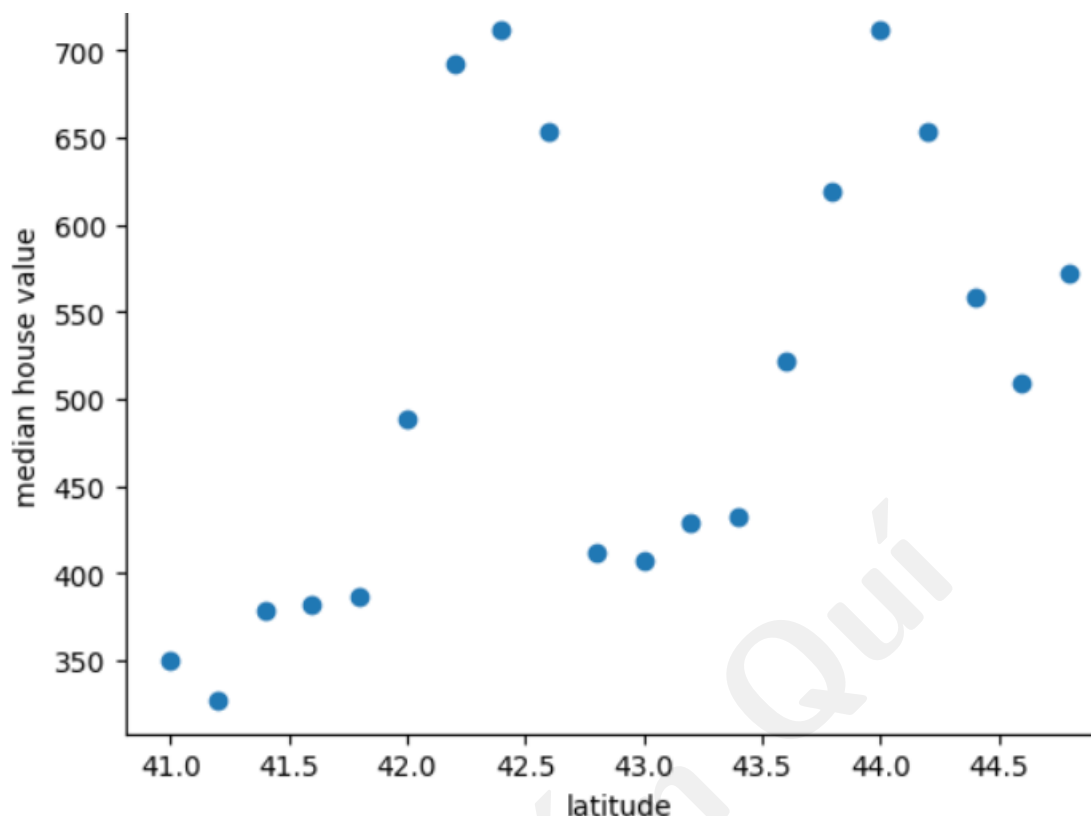
Hình 10. Cùng scatter với đường chia bin.

Mô hình học trọng số **riêng** mỗi bin.

Không nên quá nhiều bin

- Mỗi bin cần **đủ ví dụ** để học liên hệ bin–nhân. Ba bin ≥ 10 điểm/bin có thể đủ; 33 bin (mỗi nhiệt độ một bin) thì không bin nào đủ dữ liệu.
- 33 bin = 33 feature nhiệt độ — thường nên **giảm** số feature.

5.2 Bài tập: giá nhà theo vĩ độ (Freedonia)



Hình 11. Giá trung vị mỗi $0,2^\circ$ vĩ độ; hai cụm rõ (41,0–41,8 và 42,6–43,4), phần còn lại gần ngẫu nhiên.

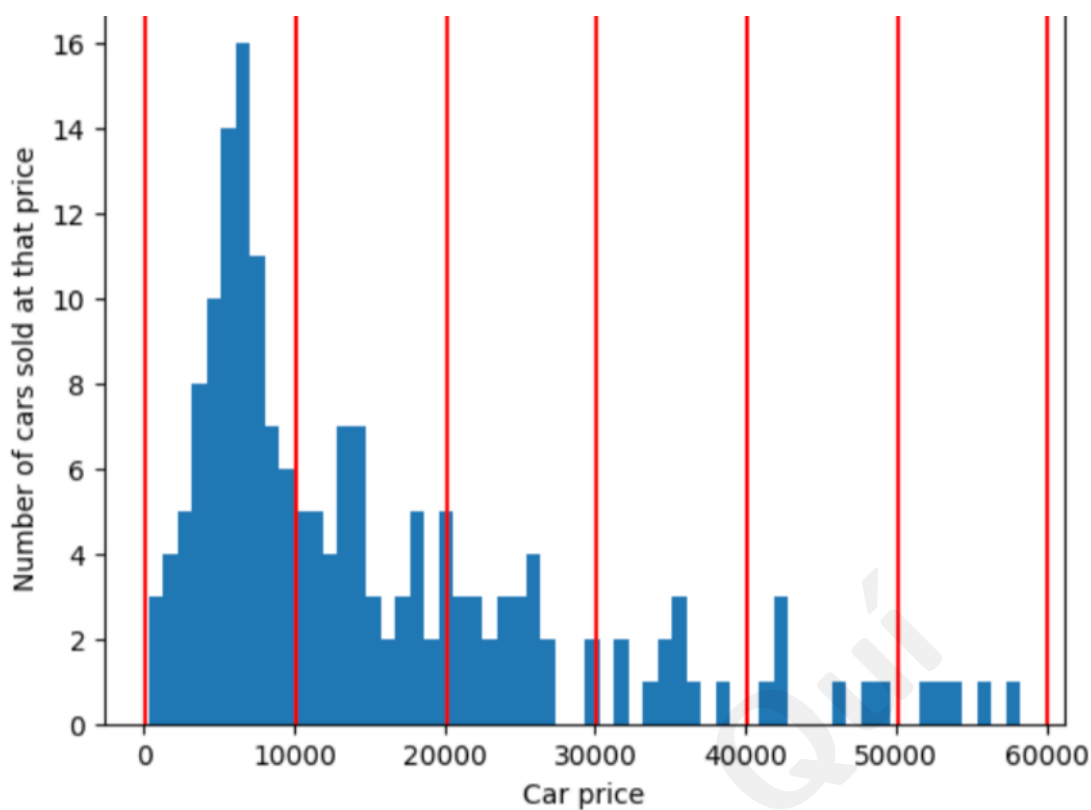
Quan hệ **phi tuyến** — float vĩ độ khó giúp dự đoán. Bucket có giúp?

Chiến lược bucket tốt nhất?

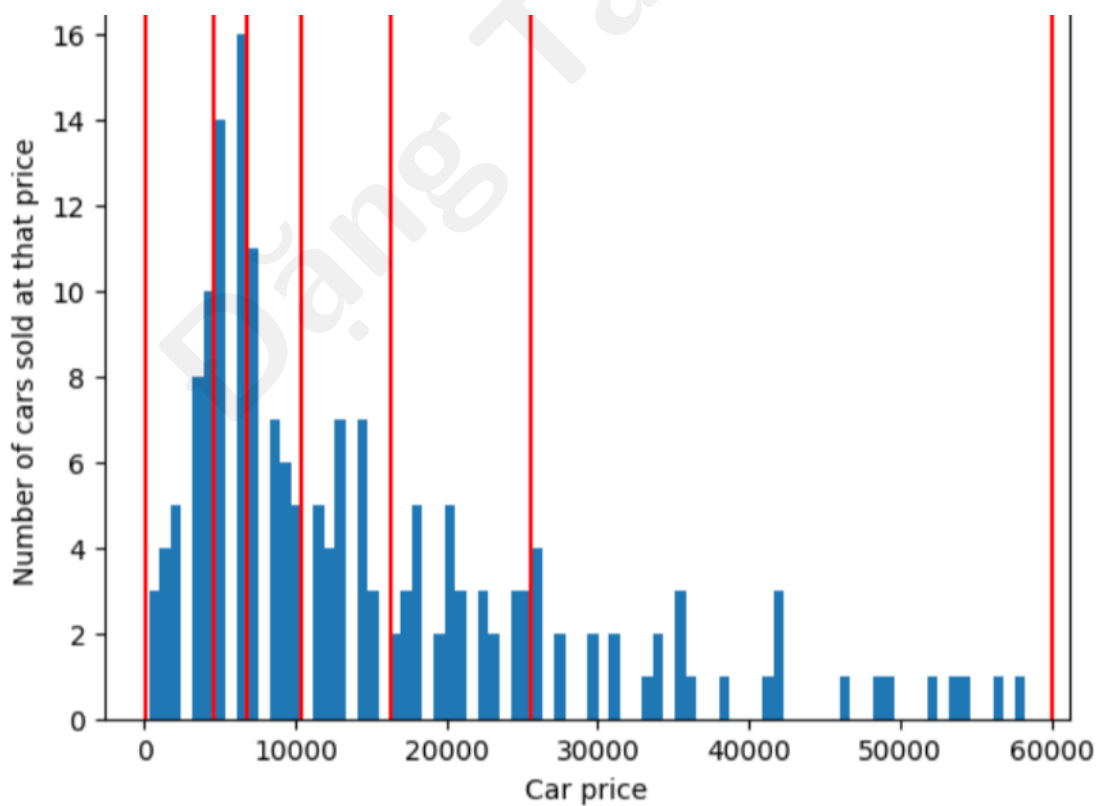
Đáp án: Không bucket. Phần lớn đồ thị ngẫu nhiên — bucket khó có trọng số dự đoán ổn. Bốn bucket (41,0–41,8, 42,0–42,6, ...) — bin 2 và 4 ít ví dụ. Mỗi $0,2^\circ$ một bucket chỉ hữu ích nếu đủ mẫu mỗi vĩ độ.

5.3 Quantile bucketing (bin theo phân vị)

Quantile bucketing đặt biên sao cho số ví dụ mỗi bin gần bằng nhau, thường che outlier.



Hình 13. Mười bucket **bằng nhau** 10 000 USD: 0–10k có hàng chục xe; 50–60k chỉ 5 xe — không đủ học bucket cao.



Hình 14. Quantile: mỗi bucket $\approx$ cùng số xe; một số bucket hẹp, một số rất rộng.

Lưu ý

Bucket **khoảng đều** phù hợp nhiều phân bố. Với dữ liệu **lệch** (skewed): thử **quantile** — khoảng đều cho nhiều không gian cho đuôi dài, nén “thân” vào một bin; quantile cho nhiều không gian cho thân, nén đuôi.

6 Scrubbing (tua / làm sạch)

Ẩn dụ táo

Cây táo có quả ngon và quả sâu. Siêu thị chỉ bán táo đẹp — giữa vườn và kệ có người loại táo hỏng hoặc phủ sáp táo cứu được. Kỹ sư ML cũng dành rất nhiều thời gian loại ví dụ xấu và “đánh bóng” ví dụ còn cứu được. Vài “táo hỏng” có thể hỏng cả dataset lớn.

Công thức**Loại vấn đề****Ví dụ**

Giá trị thiếu

Điều tra viên không ghi tuổi cư dân.

Trùng lặp

Server upload cùng log hai lần.

Ngoài miền

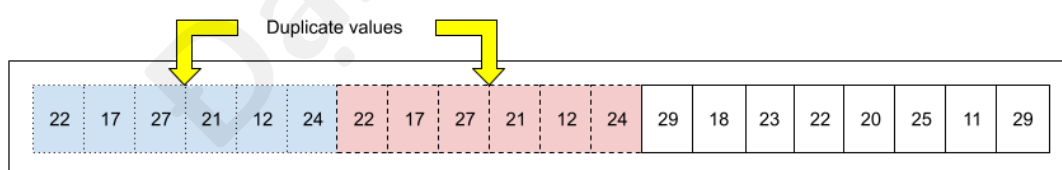
Gõ thừa một chữ số.

Nhãn sai

Ảnh sồi bị gán nhãn phong.

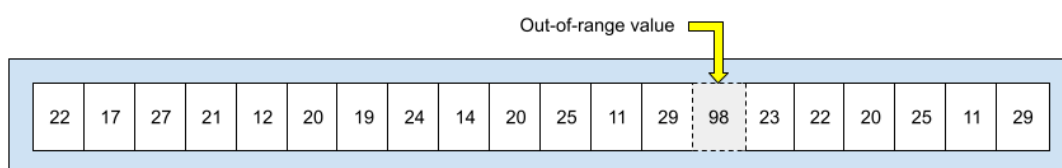
Có thể viết chương trình/script phát hiện:

- Giá trị thiếu
- Ví dụ trùng
- Feature ngoài miền cho phép



Hình 15. Sáu giá trị đầu lặp lại; tám giá trị sau không.

Nhiệt độ feature phải trong $[10, 30]^{\circ}\text{C}$ nhưng cảm biến phơi nắng tạo outlier — script phải bắt giá trị < 10 hoặc > 30 :



Hình 16. 19 giá trị trong miền, một giá trị ngoài miền.

Khi nhận do **nhều người** gán, nên thống kê xem mỗi người có bộ nhãn **tương đương** không (người chăm nhất hơn, tiêu chí khác...).

Sau khi phát hiện, thường **sửa** bằng xóa ví dụ hoặc **impute** (điền giá trị). Chi tiết: module [Đặc điểm dữ liệu](#) trong phần overfitting.

7 Tính chất của đặc trưng số tốt

Đơn vị này đã nói cách ánh xạ dữ liệu thô sang **feature vector**. **Feature** số tốt có các đặc tính sau.

7.1 Tên rõ ràng

Mỗi feature phải có **nghĩa rõ**, dễ hiểu với mọi người trong dự án.

Không nên

house_age: 851472000 — khó hiểu.

Nên

house_age_years: 27 — rõ ràng.

Lưu ý

Đồng nghiệp sẽ phản đối tên feature/nhãn khó hiểu; **mô hình không quan tâm** (nếu bạn chuẩn hóa đúng).

7.2 Kiểm tra / thử trước khi train

Outlier đủ quan trọng để nhắc lần cuối. Đôi khi **dữ liệu xấu** (không phải kỹ thuật xấu) gây giá trị lạ:

- Không nên: user_age_in_years: 224
- Nên: user_age_in_years: 24 — có thể do nguồn không kiểm tra.

Hãy kiểm tra dữ liệu!

7.3 Hợp lý (sensible) — tránh “magic value”

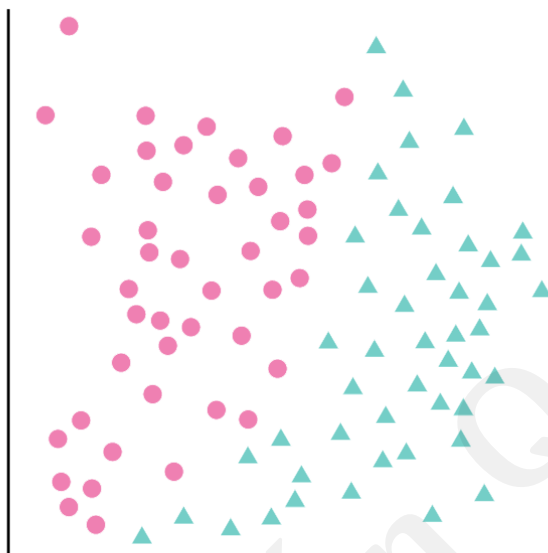
Magic value là giá trị đặc biệt cắt đứt tính liên tục của feature. Ví dụ watch_time_in_seconds từ 0–30 nhưng -1 = “không đo”:

- Không nên: watch_time_in_seconds: -1 — mô hình phải đoán “xem phim ngược thời gian”.
- Nên: tách feature Boolean is_watch_time_in_seconds_defined; ví dụ 4.82, True hoặc 0, False.

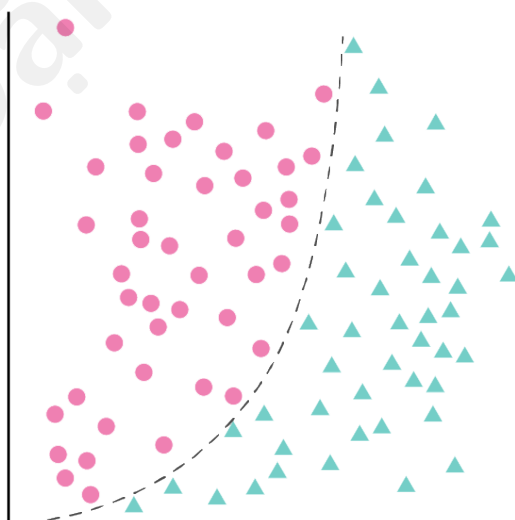
Với **feature rời rạc** như product_category thuộc tập hữu hạn, khi thiếu giá trị thêm **giá trị mới** trong tập, ví dụ {0: electronics, 1: books, 2: clothing, 3: missing_category} — mô hình học trọng số riêng cho “thiếu”.

8 Biến đổi đa thức (polynomial transforms)

Khi có **kiến thức miền** cho rằng một biến liên quan bình, lập, hoặc lũy thừa của biến khác, có thể tạo **synthetic feature** từ feature số có sẵn.



Hình 17. Hai lớp không tách được bằng đường thẳng.



Hình 18. Đường cong $y = x^2$ tách hai lớp.

Mô hình tuyến tính một feature x_1 :

$$y = b + w_1x_1$$

Thêm feature thì thêm $w_2x_2, w_3x_3 \dots$ **Gradient descent** tìm **trọng số** w_i tối thiểu mất mát. Dữ liệu trên không tách được bằng đường thẳng — làm sao?

Định nghĩa $x_2 = x_1^2$ (biến đổi đa thức, xử lý như feature bình thường):

$$y = b + w_1x_1 + w_2x_2$$

Vẫn là **hồi quy tuyến tính**; gradient descent như cũ — nhưng ranh giới có dạng $y = b + w_1x + w_2x^2$.

Thường nhân feature với chính nó (lũy thừa). Trong vật lý nhiều quan hệ có bình phương: gia tốc trọng trường, suy giảm ánh sáng/âm thanh, năng lượng thế đàn hồi.

Lưu ý

Nếu biến đổi làm đổi **quy mô** feature, nên thử **chuẩn hóa sau** biến đổi (xem mục Chuẩn hóa).

Liên quan dữ liệu danh mục

Feature cross trên dữ liệu danh mục thường tổng hợp **hai feature khác nhau** — xem module categorical.

9 Kết luận

Động cơ học – Tại sao cần khái niệm này?

Sức khỏe mô hình ML do **dữ liệu** quyết định. Dữ liệu tốt $\Rightarrow$ mô hình phát triển; dữ liệu rác $\Rightarrow$ dự đoán vô giá trị.

Thực hành tốt với dữ liệu số

- Mô hình tương tác với **feature vector**, không phải bảng **dataset** thô.
- **Chuẩn hóa** hầu hết feature số.
- Chiến lược chuẩn hóa đầu không hiệu quả $\Rightarrow$ thử cách khác.
- **Binning** đôi khi tốt hơn chuẩn hóa.
- Viết **test xác minh** theo kỳ vọng nghiệp vụ: vĩ độ $|\text{lat}| \leq 90$; nếu chỉ Florida thì $\text{lat} \in [24, 31]$.
- Scatter plot, histogram — tìm bất thường.
- Thống kê trên **cả dataset** và **tập con** — thống kê gộp đôi khi che lỗi ở phần nhỏ.
- **Ghi chép** mọi biến đổi dữ liệu.